

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 5: Physical Computing with Raspberry PI

Name of Student: ..[Bushra Nazir](#).....

Registration No.:[1071-F22](#).....

Date of Experiment:[30-4-2026](#).....

Submitted To:[Engr.Asad](#).....

Objectives

- To introduce students to the GPIO capabilities of Raspberry Pi for interfacing and controlling electronic hardware components.
- To develop hands-on skills in reading sensor data and controlling actuators using Python-based event-driven programming.
- To build a strong foundation for integrating AI/IoT applications with physical environments using basic electronics.

Equipment Needed

- Raspberry Pi (any model with GPIO): Quantity 1 per group
- Breadboard: Quantity 1
- LEDs (Red, Green, Blue): Quantity 4–6
- 220Ω Resistors: Quantity 6–8
- Push Buttons: Quantity 2–4
- Jumper Wires (M-M, F-M): Quantity 20+
- Buzzer (Active or Passive): Quantity 1
- PIR Sensor: Quantity 1
- Ultrasonic Sensor (HC-SR04): Quantity 1
- DHT11 Sensor: Quantity 1
- Servo Motor (SG90): Quantity 1
- DC Motor with Driver (L298N): Quantity 1
- Potentiometer: Quantity 1
- OLED Display (I2C, 0.96"): Quantity 1
- 10kΩ Resistors: Quantity 2–4
- External 5V Power Supply: Quantity 1

Introduction

This lab manual is designed to provide hands-on experience in physical computing using the Raspberry Pi along with a variety of essential electronic components. Through a series of structured experiments, students will learn how to interface sensors, actuators, and output devices with the Raspberry Pi's GPIO pins to build interactive and intelligent systems. The equipment includes LEDs, push buttons, buzzers, sensors (such as PIR, ultrasonic, and DHT11), motors, displays, and other core components necessary for embedded development. In addition to foundational physical computing tasks, this manual introduces students to the practical deployment and verification of machine learning algorithms at the edge. Using the Raspberry Pi as a gateway between hardware and AI, students will explore real-world applications such as motion detection, gesture control, and environmental monitoring enhanced by ML models. This lab bridges electronics, programming, and artificial intelligence, enabling students to build smart systems that sense, process, and act autonomously.

GPIO pins

One powerful feature of the Raspberry Pi is the row of GPIO pins along the top edge of the board. GPIO stands for General-Purpose Input/Output. These pins are a physical interface between the Raspberry Pi and the outside world. At the simplest level, you can

think of them as switches that you can turn on or off (input) or that the Pi can turn on or off (output).

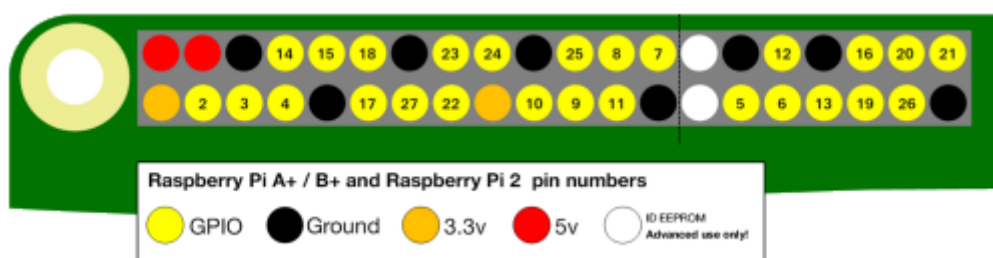
The GPIO pins allow the Raspberry Pi to control and monitor the outside world by being connected to electronic circuits. The Pi is able to control LEDs, turning them on or off, run motors, and many other things. It's also able to detect whether a switch has been pressed, the temperature, and light. We refer to this as physical computing.

There are 40 pins on the Raspberry Pi (26 pins on early models), and they provide various different functions.

If you have a RasPiO pin label, it can help to identify what each pin is used for. Make sure your pin label is placed with the keyring hole facing the USB ports, pointed outwards.



If you don't have a pin label, then this guide can help you to identify the pin numbers:

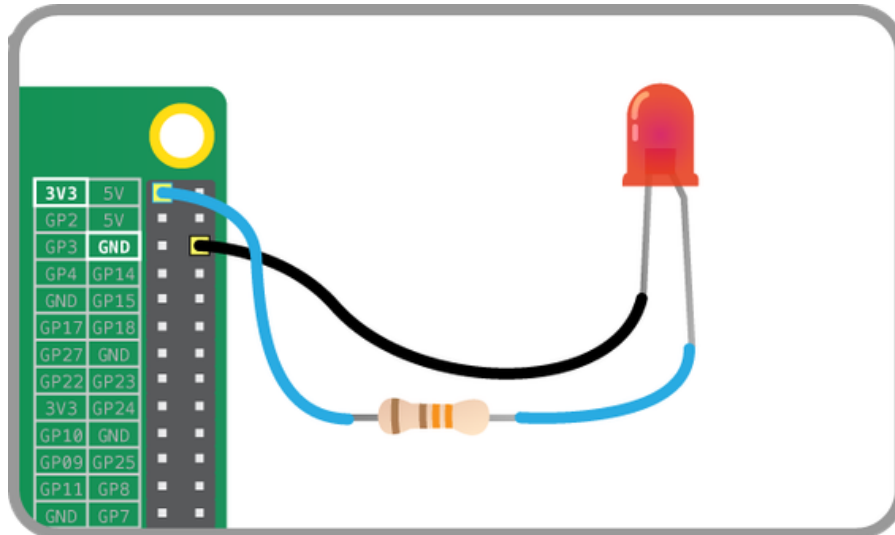


You'll see pins labelled as 3V3, 5V, GND and GP2, GP3, etc:

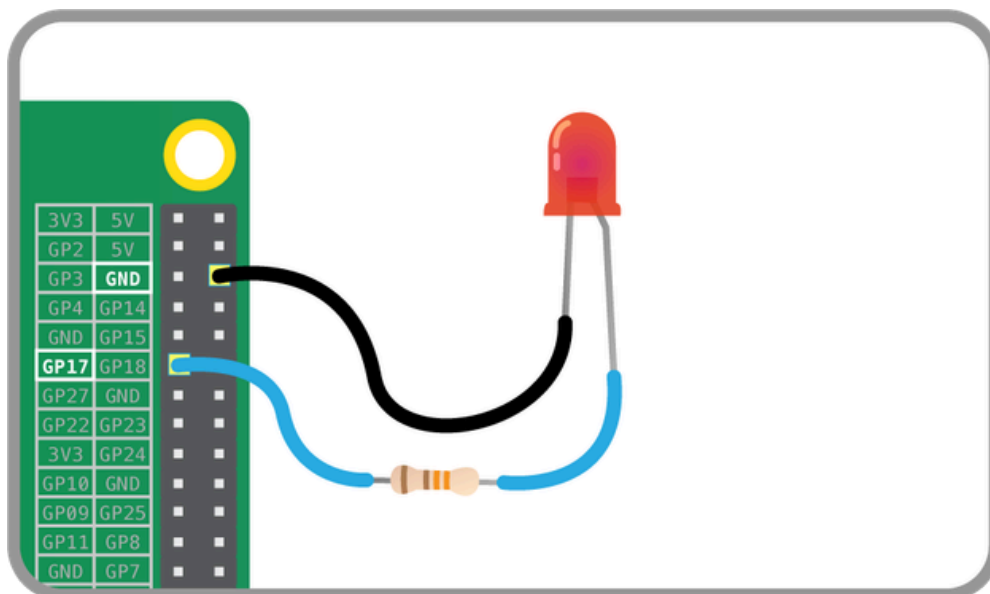
WARNING: If you follow the instructions, then playing about with the GPIO pins is safe and fun. Randomly plugging wires and power sources into your Pi, however, may destroy it, especially if using the 5V pins. Bad things can also happen if you try to connect things to your Pi that use a lot of power; LEDs are fine, motors are not.

Task-1 Lighting an LED

LEDs are delicate little things. If you put too much current through them they will pop (sometimes quite spectacularly). To limit the current going through the LED, you should always use a resistor in series with it. Try connecting the long leg of an LED to the Pi's 3V3 and the short leg to a GND pin. The resistor can be anything over about 50Ω.



The LED should light up. It will always be on, because it's connected to a 3V3pin, which is itself always on. Now try moving it from 3V3 to GPIO pin 17:

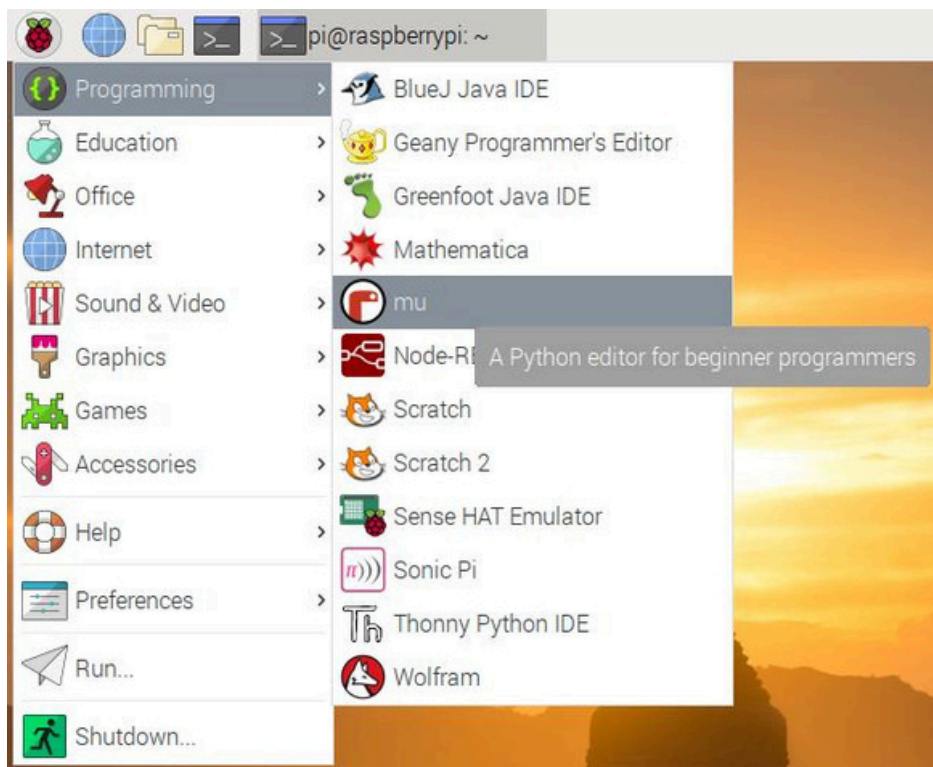


The LED should now turn off, but now it's on a GPIO pin, and can therefore be controlled by code.

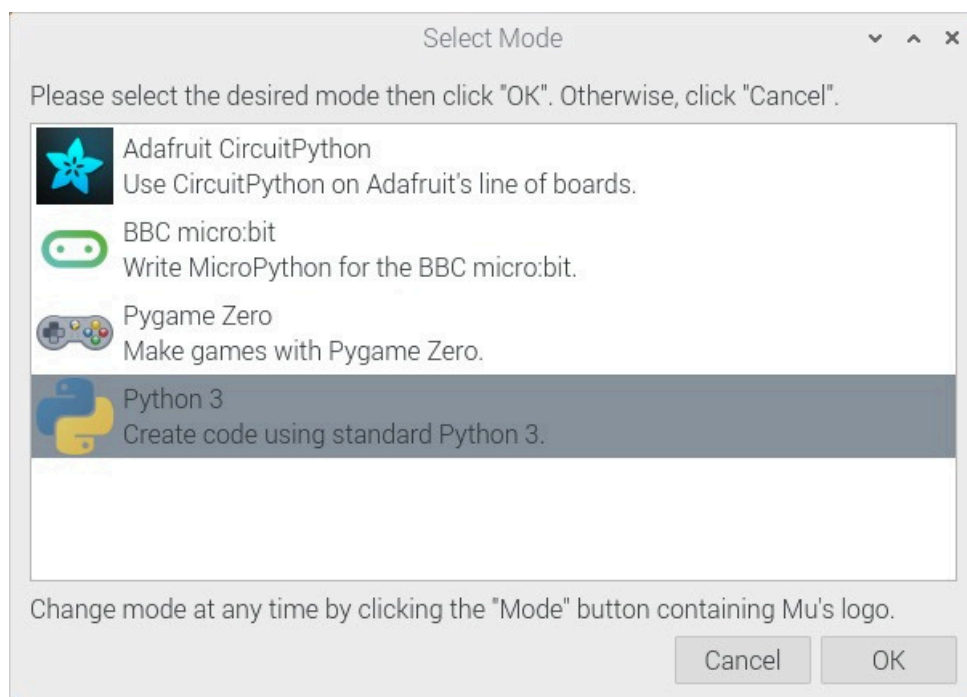
Switching an LED on and Off

GPIO Zero is a new Python library which provides a simple interface to everyday GPIO components. It comes installed by default in Raspbian.

- o Open Mu. By going to the **Programming** menu and click on Mu.



- ✓ Then choose the mode in which you want to use Mu. Choose **Python 3** if you are creating a new Python script.



- ✓ You can switch an LED on and off by typing commands directly into the REPL. Click on the REPL button in the menu bar.
- ✓ First import the GPIO Zero library, and tell the Pi which GPIO pin you are using - in this case pin 17.

```
from gpiozero import LED
led = LED(17)
```

✓ Press **Enter** on the keyboard.

✓ To make the LED switch on, type the following and press **Enter**:

```
led.on()
```

To make it switch off you can type:

```
led.off()
```

Flashing an LED

With the help of the time library and a little loop, you can make the LED flash.

- ✓ Create a new file by clicking **New**.
- ✓ Save the new file by clicking **Save**. Save the file as `gpio_led.py`.
- ✓ Enter the following code to get started:

```
from gpiozero import LED
from time import sleep

led = LED(17)

while True:
    led.on()
    sleep(1)
    led.off()
    sleep(1)
```

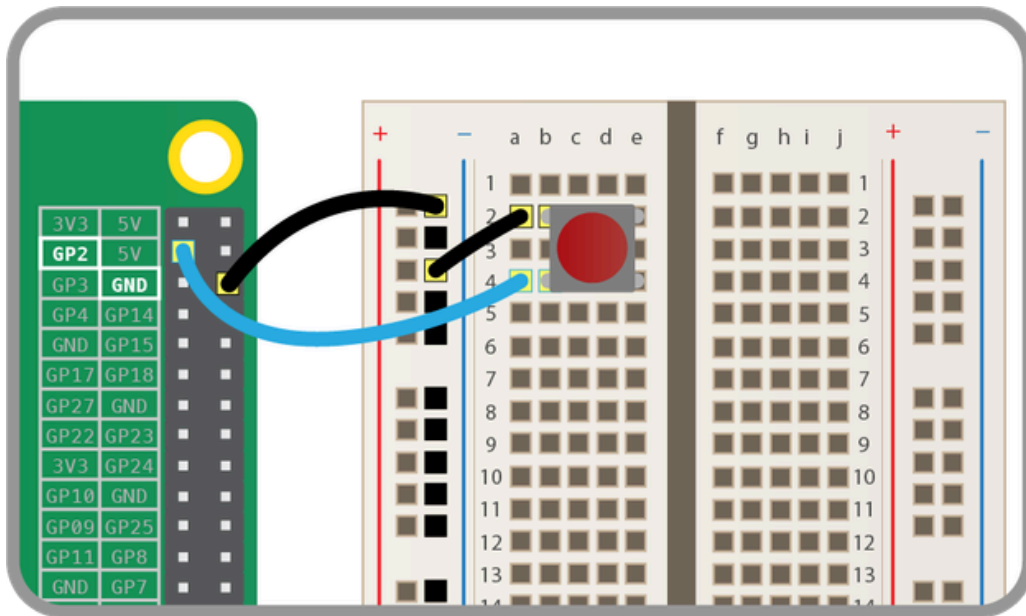
✓ Save the file and run the code with by clicking on **Run**.

✓ The LED should be flashing on and off. To exit the program click **Stop**.

Using buttons to get input

Now you're able to control an output component (an LED), let's connect and control an input component: a button.

- ✓ Connect a button to another GND pin and GPIO pin 2, like this:



- ✓ Create a new file by clicking **New**.
- ✓ Save the new file by clicking **Save**. Save the file as `gpio_button.py`.
- ✓ This time you'll need the Button class, and to tell it that the button is on pin 2. Write the following code in your new file:

```
from gpiozero import Button
button = Button(2)
button.wait_for_press()
print('You pushed me')
```

- ✓ Save and run the code.
- ✓ Press the button and your text will appear.

Manually controlling the LED

- ✓ You can now combine your two programs written so far to control the LED using the button.
- ✓ Create a new file by clicking **New**.
- ✓ Save the new file by clicking **Save**.
- ✓ Now write the following code

```

from gpiozero import LED, Button
from time import sleep
led = LED(17)
button = Button(2)
button.wait_for_press()
led.on()
sleep(3)
led.off()

```

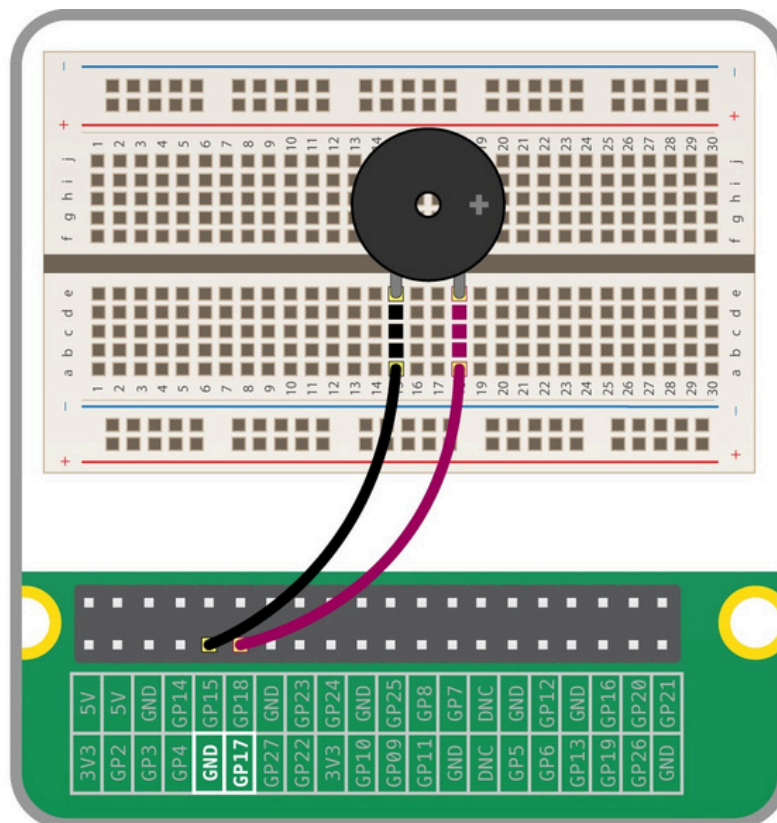
✓ Save and run your program. When you push the button the LED should come on for three seconds.

Task 2 Using a buzzer

There are two main types of buzzers: *active* and *passive*. A *passive* buzzer emits a tone when a voltage is applied across it. It also requires a specific signal to generate a variety of tones. The *active* buzzers are a lot simpler to use, so we will be using an active buzzer for this task.

Connecting a buzzer

An active buzzer can be connected just like an LED, but as they are a little more robust, you won't need a resistor to protect them. Set up the circuit as shown below:



Add **Buzzer** to the **from gpiozero import...** line:

```
from gpiozero import Buzzer
from time import sleep
```

Add a line below your creation of `button` and `lights` to add a `Buzzer` object:

```
buzzer = Buzzer(17)
```

In GPIO Zero, a `Buzzer` works exactly like an `LED` so try adding a `buzzer.on()` and `buzzer.off()` into your loop:

```
while True:
    buzzer.on()
    sleep(1)
    buzzer.off()
    sleep(1)
```

A `Buzzer` has a `beep()` method which works like an `LED`'s `blink`. Try it out:

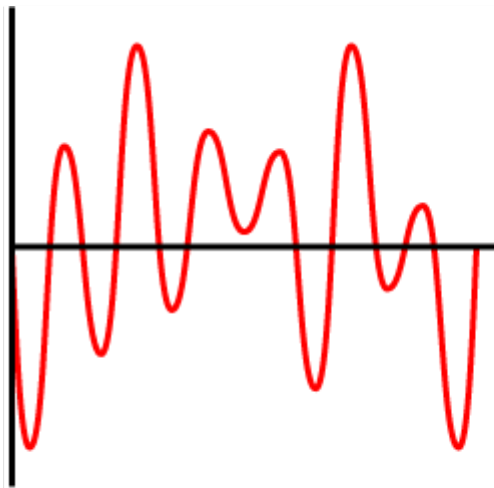
```
while True:
    buzzer.beep()
```

Task 3: Using a light-dependent resistor

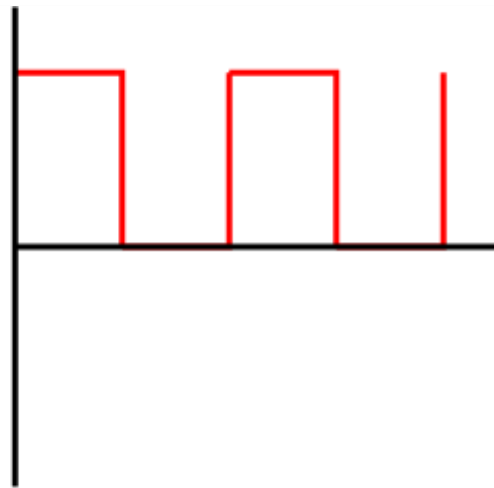
Analogue inputs

In the world of electrical engineering, there are two type of input and output (I/O): analogue and digital. Digital I/O is fairly easy to understand; it's either *on* or *off*, *1* or *0*. When talking about voltages and the Raspberry Pi, any input that is approximately below 1.8V is considered *off* and anything above 1.8V is considered *on*. For output, 0V is off and 3.3V is on.

Analogue I/O is a little trickier. With an analogue input, we can have a range of voltages from 0V up to 3.3V, and the Raspberry Pi is unable to detect exactly what that voltage is.



Analogue inputs could be given a range of voltages, anywhere from 0V up to 3.3V.

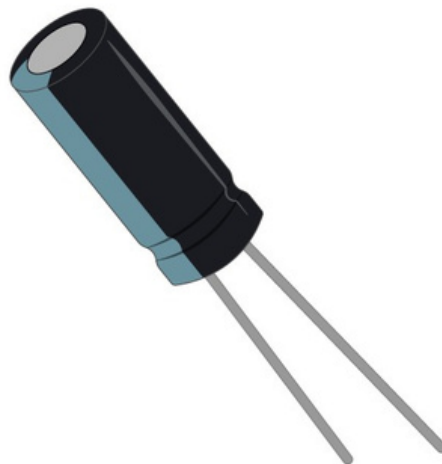


Digital inputs are either on or off. Any voltage above 1.8V is considered on, and below that is considered off.

How, then, can we use a Raspberry Pi to determine the value of an analogue input, if it can only tell when the voltage to a GPIO pin goes above 1.8V?

Using a capacitor for analogue inputs

Capacitors are electrical components that store charge.



When current is fed into a capacitor, it will begin to store charge. The voltage across the capacitor will start off low, and increase as the charge builds up. By putting a resistor in series with the capacitor, you can slow the speed at which it charges. With a high resistance, the capacitor will charge slowly, whereas a low resistance will let it charge quickly.

If you time how long it takes the capacitor's voltage to get over 1.8V (or be *on*), you can work out the resistance of the component in series with it. Many, but not all, capacitors are polarized which means they have a positive and a negative leg. In this case, the negative leg is shorter and should be marked with a '-' symbol.

Light-dependent resistors

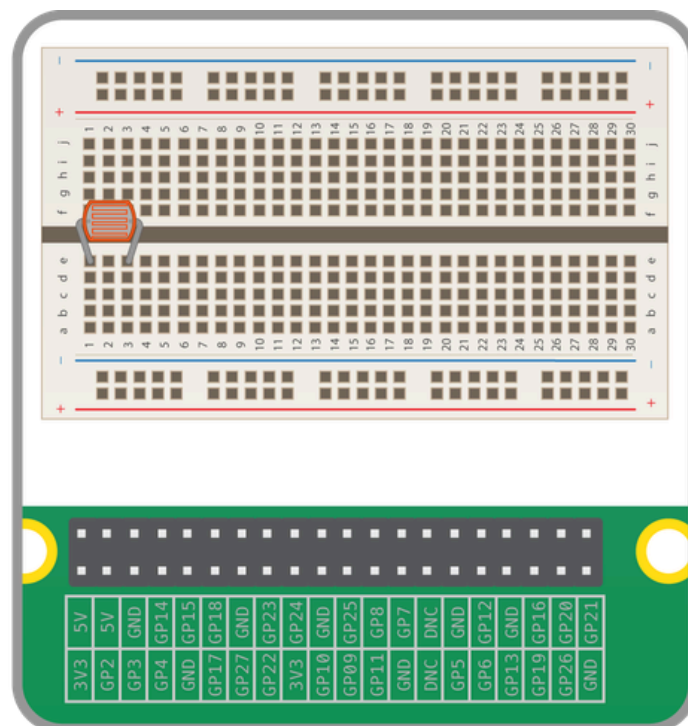
An LDR (sometimes called a photocell) is a special type of resistor.



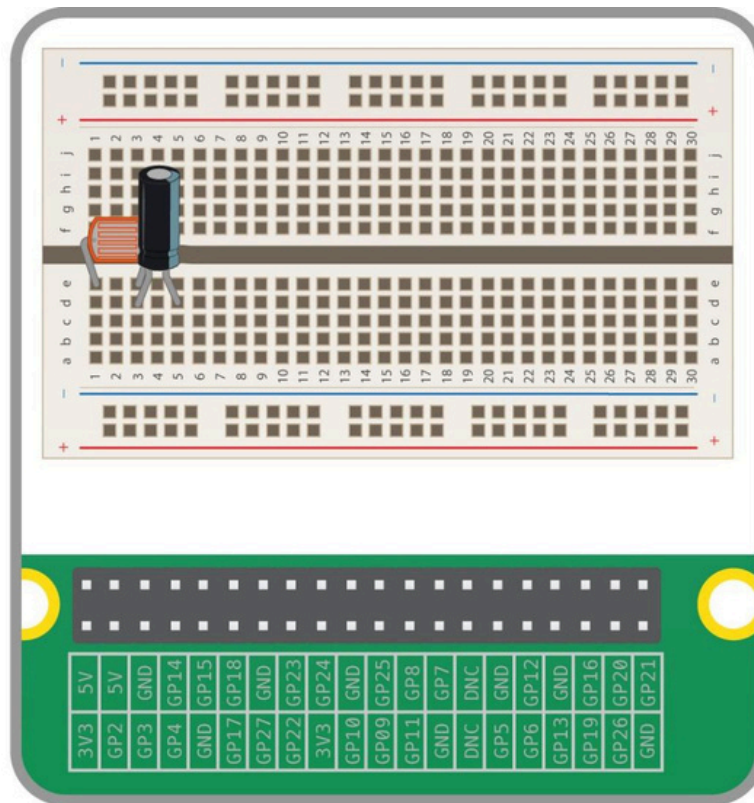
When light hits the LDR, its resistance is very low, but when it's in the dark its resistance is very high. By placing a capacitor in series with an LDR, the capacitor will charge at different speeds depending on whether it's light or dark.

Creating a light-sensing circuit

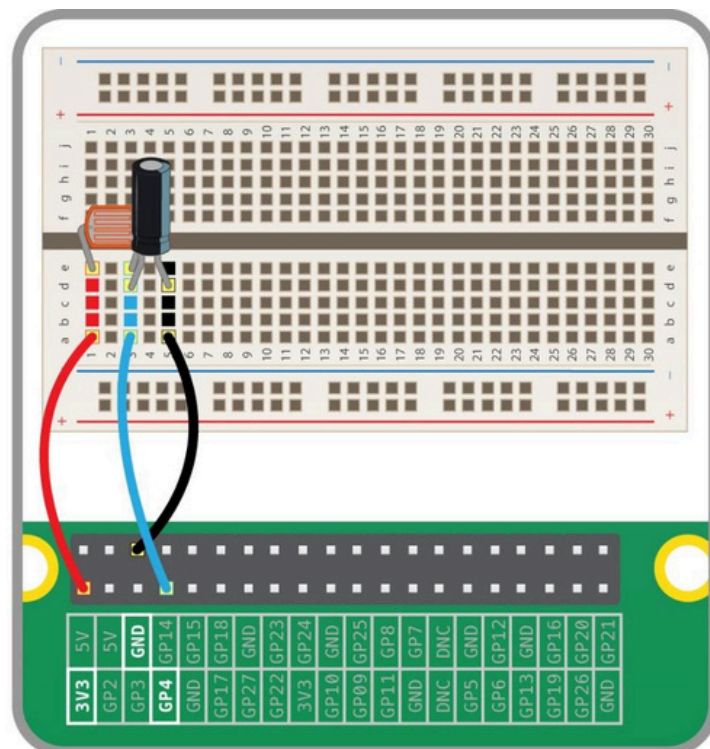
✓ Place an LDR into your breadboard, as shown below:



- ✓ Now place a capacitor in series with the LDR. As the capacitor is a polar component, you must make sure the positive, long leg is on the same track as the LDR leg.



- o Finally, add jumper leads to connect the two components to your Raspberry Pi.



Coding a light sensor

The gpiozero library will handle the timing of the capacitor's charging and discharging for light sensor circuit. Use the following code to set up the light sensor:

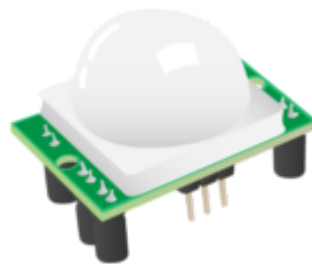
```
from gpiozero import LightSensor, Buzzer
ldr = LightSensor(4) # alter if using a different pin
while True:
    print(ldr.value)
```

Run this code, then cover the LDR with your hand and watch the value change. Try shining a strong light onto the LDR.

Task 4- Using a PIR sensor

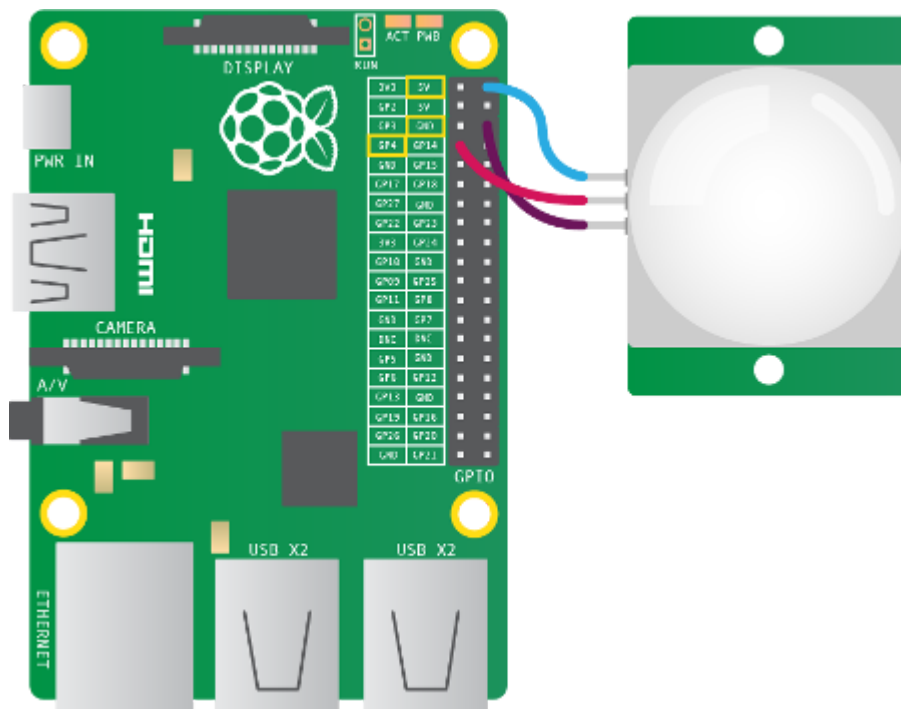
Humans and other animals emit radiation all the time. This is nothing to be concerned about, though, as the type of radiation we emit is infrared radiation (IR), which is pretty harmless at the levels at which it is emitted by humans. In fact, all objects at temperatures above absolute zero (-273.15C) emit infrared radiation.

A PIR sensor detects changes in the amount of infrared radiation it receives. When there is a significant change in the amount of infrared radiation it detects, then a pulse is triggered. This means that a PIR sensor can detect when a human (or any animal) moves in front of it.



Wiring a PIR sensor

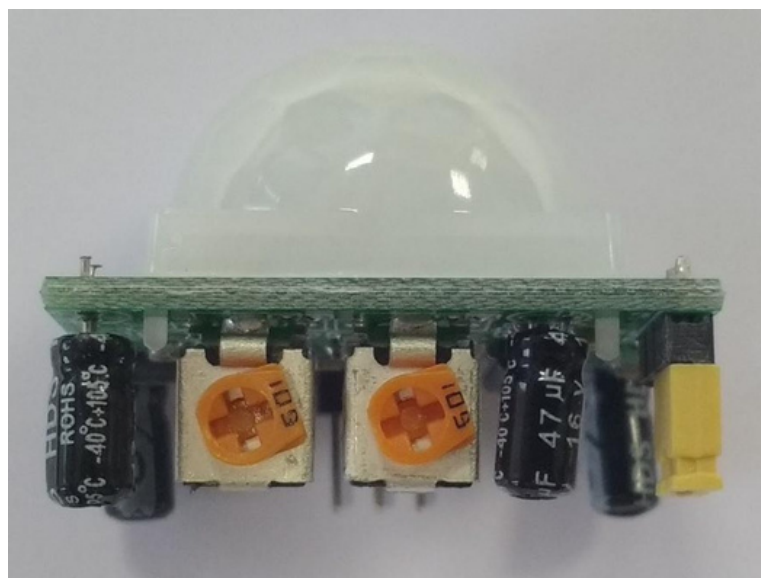
The pulse emitted when a PIR detects motion needs to be amplified, and so it needs to be powered. There are three pins on the PIR: they should be labelled **Vcc**, **Gnd**, and **Out**. These labels are sometimes concealed beneath the Fresnel lens (the white cap), which you can temporarily remove to see the pin labels.



1. As shown above, the **Vcc** pin needs to be attached to a **5V** pin on the Raspberry Pi.
2. The **Gnd** pin on the PIR sensor can be attached to **any** groundpin on the Raspberry Pi.
3. Lastly, the **Out** pin needs to be connected to **any** of the GPIO pins.

Tuning a PIR

Most PIR sensors have two potentiometers on them. These can control the sensitivity of the sensors, and also the period of time for which the PIR will signal when motion is detected.



In the image above, the potentiometer on the right controls the sensitivity, and the potentiometer on the left controls the timeout. Here, both are turned fully anti-clockwise, meaning that the sensitivity and timeout are at their lowest. When the timeout is turned fully anti-clockwise, the PIR will output a signal for about 2.5 seconds, whenever motion is detected. If the potentiometer is turned fully clockwise, the output signal will last for around 250 seconds. When tuning the sensitivity, it is best to have the timeout set as low as possible.

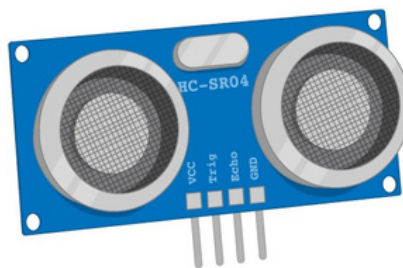
Detecting motion

You can detect motion with the PIR using the code below:

```
from gpiozero import MotionSensor
pir= MotionSensor(4)
while True:
    pir.wait_for_motion()
    print("You moved")
    pir.wait_for_no_motion()
```

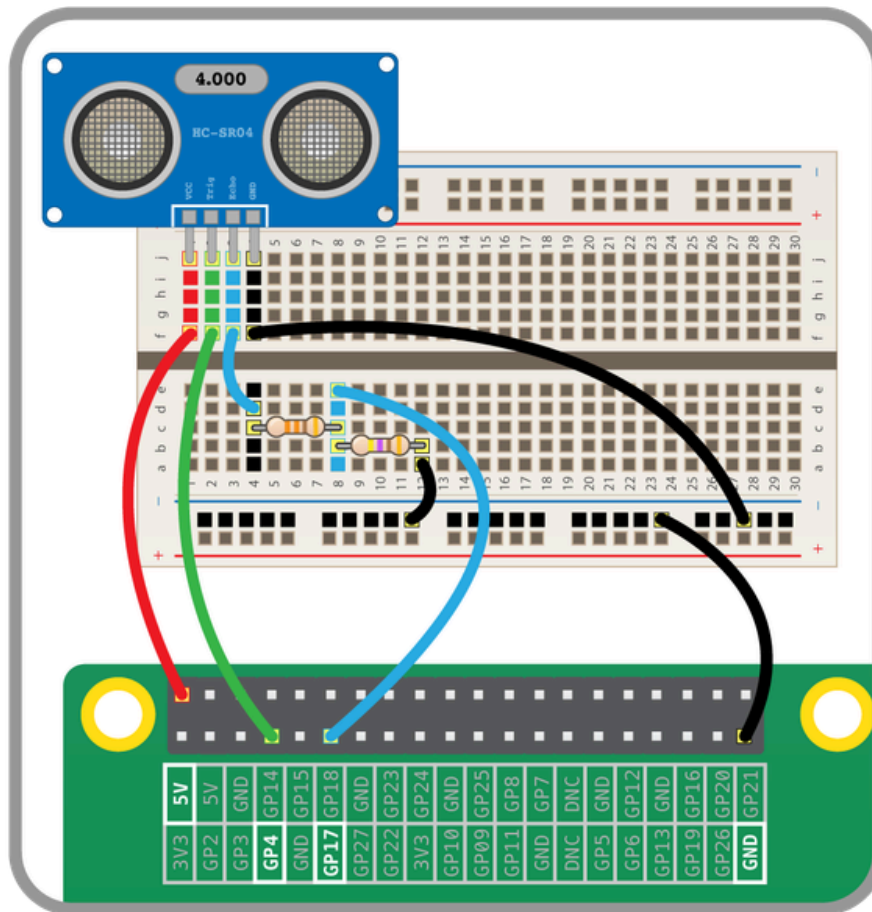
Task 5 Using an ultrasonic distance sensor

In air, sound travels at a speed of 343 metres per second. An ultrasonic distance sensor sends out pulses of ultrasound which are inaudible to humans, and detects the echo that is sent back when the sound bounces off a nearby object. It then uses the speed of sound to calculate the distance from the object.



Wiring

The circuit connects to two GPIO pins (one for echo, one for trigger), the ground pin, and a 5V pin. You'll need to use a pair of resistors (330Ω and 470Ω) as a potential divider:



Code

To use the ultrasonic distance sensor in Python, you need to know which GPIO pins the echo and trigger are connected to.

✓ Use the following code to read the DistanceSensor.

```
from gpiozero import DistanceSensor
ultrasonic = DistanceSensor(echo=17, trigger=4)
while True:
    print(ultrasonic.distance)
```

- ✓ The value should get smaller the closer your hand is to the sensor. Press **Stop** to exit the loop.

Ranges

As well as being able to see the distance value, you can also get the sensor to do things when the object is in or out of a certain range.

- ✓ Use a loop to print different messages when the sensor is in range or out of range:

```
while True:
    ultrasonic.wait_for_in_range()
    print("In range")
    ultrasonic.wait_for_out_of_range()
    print("Out of range")
```

- ✓ Now wave your hand in front of the sensor; it should switch between showing the message "In range" and "Out of range" as your hand gets closer and further away from the sensor. See if you can work out the point at which it changes.

- ✓ The default range threshold is 0.3m. This can be configured when the sensor is initiated:

```
ultrasonic = DistanceSensor(echo=17, trigger=4, threshold_distance=0.5)
```

Alternatively, this can be changed after the sensor is created, by setting the `threshold_distance` property:

```
ultrasonic.threshold_distance = 0.5
```

- ✓ Try the previous loop again and observe the new range threshold.
- ✓ The `wait_for` functions are **blocking**, which means they halt the program until they are triggered. Another way of doing something when the sensor goes in and out of range is to use `properties`, which can be used to trigger actions in the background while other things are happening in the code.
 - ✓ First, you need to create a function for what you want to happen when the sensor is in range:

```
def hello():  
    print("Hello")
```

Then set `ultrasonic.when_in_range` to the name of this function:

```
ultrasonic.when_in_range = hello
```

Add another function for when the sensor goes out of range

```
def bye():  
    print("Bye")  
    ultrasonic.when_out_of_range = bye
```

Now these triggers are set up, you'll see "hello" printed when your hand is in range, and "bye" when it's out of range.

✓ You may have noticed that the sensor distance stopped at 1 metre. This is the default maximum and can also be configured on setup:

```
ultrasonic = DistanceSensor(echo=17, trigger=4, max_distance=2)
```

Or after setup:

```
ultrasonic.max_distance = 2
```

✓ Try different values of `max_distance` and `threshold_distance`

Task 6 Analogue inputs

The Raspberry Pi's GPIO pins are digital pins, so you can only set outputs to high or low, or read inputs as high or low. However, using an ADC chip (Analogue-to-Digital converter), you can read the value of analogue input devices such as potentiometers.

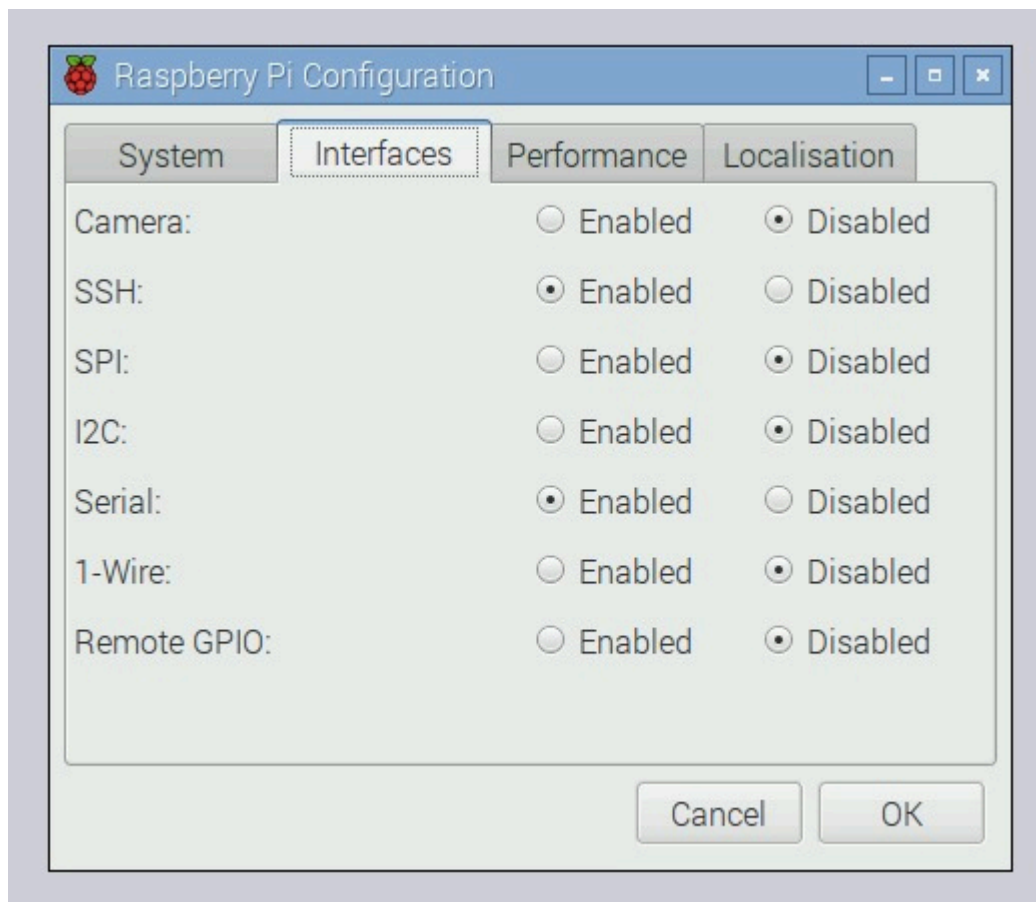
SPI

The analogue values are communicated to the Pi using the SPI protocol. While this will work in GPIO Zero out of the box, you may get better results if you enable full SPI support.

✓ Open a terminal window and install the `spidev` package:

```
sudo apt-get install python3-spidev python-spidev
```

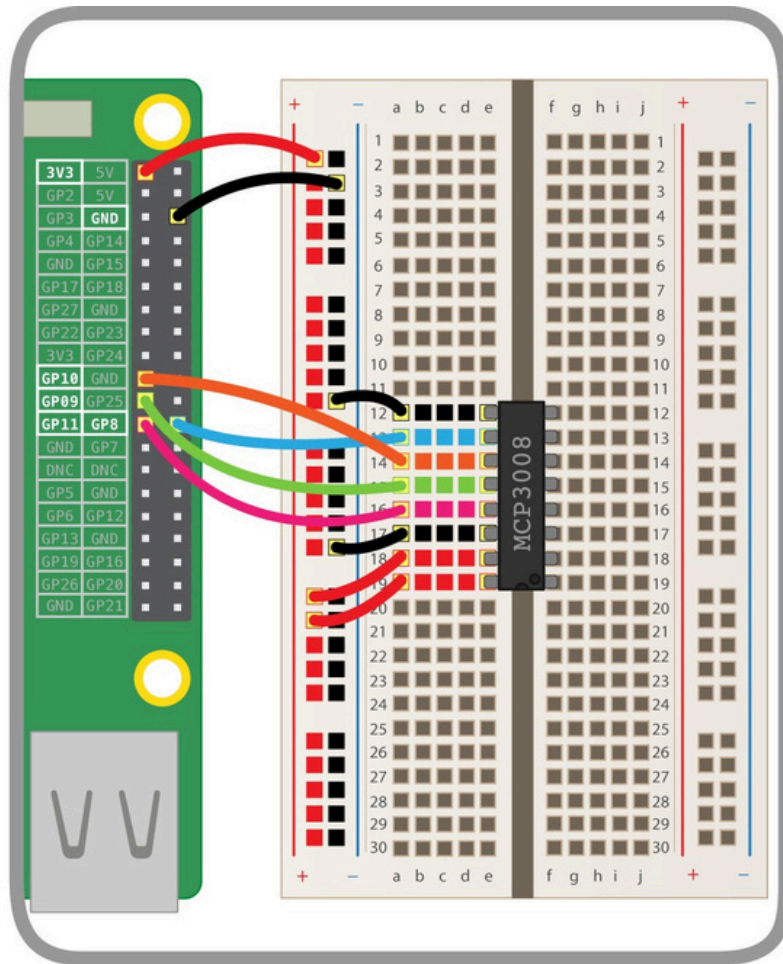
✓ Open the **Raspberry Pi Configuration** dialogue from the main menu and enable **SPI** in the **Interfaces** tab:



✓ Click **OK** and reboot the Pi.

Wiring the ADC (MCP3008)

The MCP3008 is an ADC providing eight input channels. The eight connectors on one side are connected to the Pi's GPIO pins, and the other eight are available to connect analogue input devices to read their values. Place the MCP3008 chip on a breadboard and carefully wire it up as shown in the following diagram. You should see a small notch, or dot, in one end of the chip. In the diagram, this end of the chip is aligned with column **19** on the breadboard.



Alternatively, you could use the Analog Zero board, which provides the MCP3008 chip on a handy add-on board to save you from the complicated wiring.

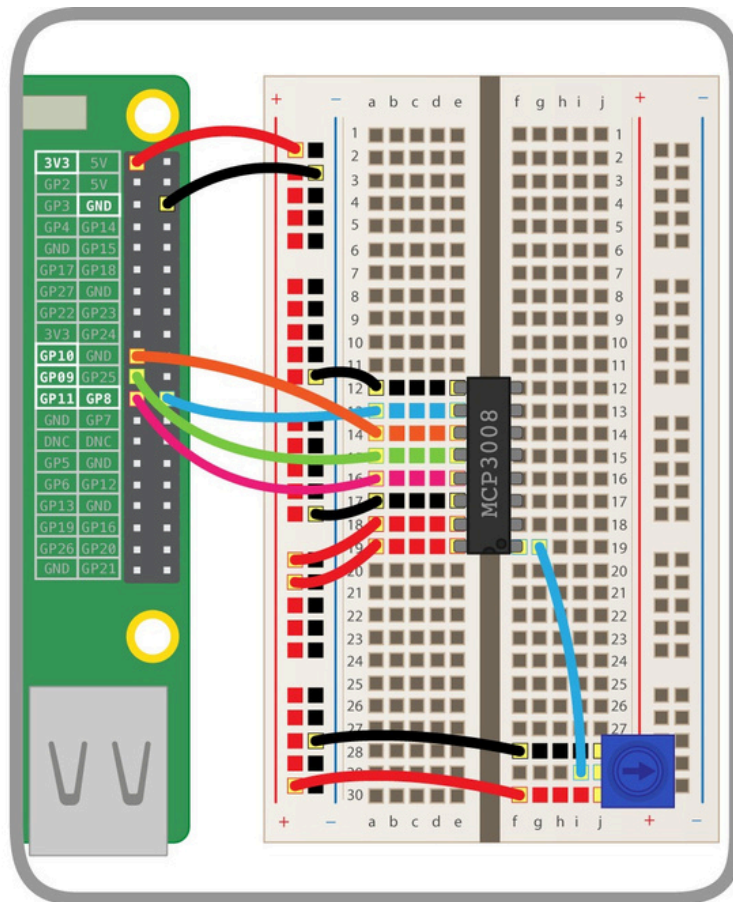
Add a potentiometer

Now that the ADC is connected to the Pi, you can wire devices up to the input channels. A potentiometer is a good example of an analogue input device: it's simply a variable resistor, and the Pi reads the voltage (from 0V to 3.3V).



A potentiometer's pins are ground, data, and 3V3. This means you connect it to ground and a supply of 3V3, and read the actual voltage from the middle pin.

- ✓ Place a potentiometer on the breadboard and wire one side to the ground rail, the other to the 3V3 rail, and the middle pin to the first input channel as shown:



Code

Now your potentiometer is connected and its value can be read from Python!

- ✓ Open **Mu** from the main menu.
- ✓ Start by importing the `MCP3008` class from the GPIO Zero library:

```
from gpiozero import MCP3008
```

- ✓ Create an object representing your analogue device:

```
pot = MCP3008(0)
```

- ✓ Note the 0 represents the ADC's channel 0. There are 8 channels (0 to 7), and you're using the first one.
- ✓ Try to read its value:

- ✓ Run your code. You should see a number between 0 and 1. This represents how far the dial is turned.
- ✓ Now read the value in a loop:

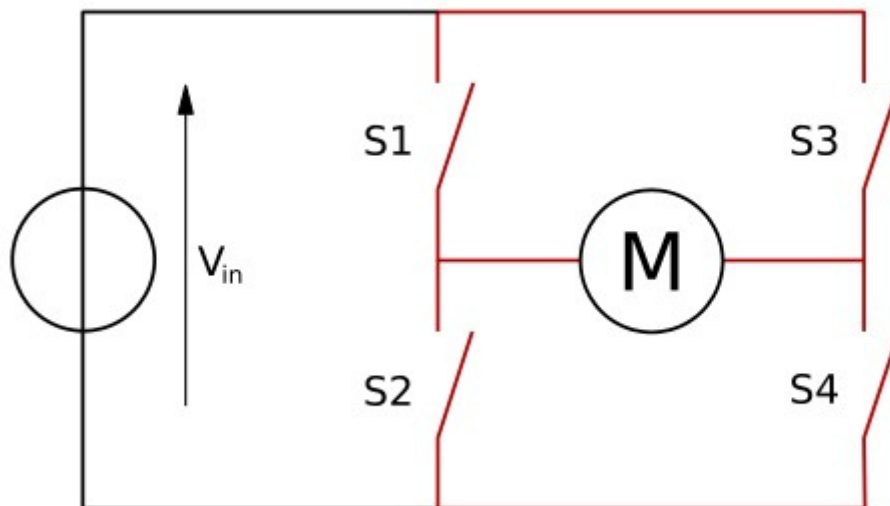
```
while True:  
    print(pot.value)
```

Task 7-Using motors

Motors are great for physical computing: they allow you to turn a wheel forwards and backwards, or make something spin around. A motor can't be controlled directly from the Raspberry Pi's GPIO pins, because it needs a variable supply of 5 volts. This means you need to power it separately. However, motor controller add-on boards called H- bridge can be used to provide this functionality.

H bridge

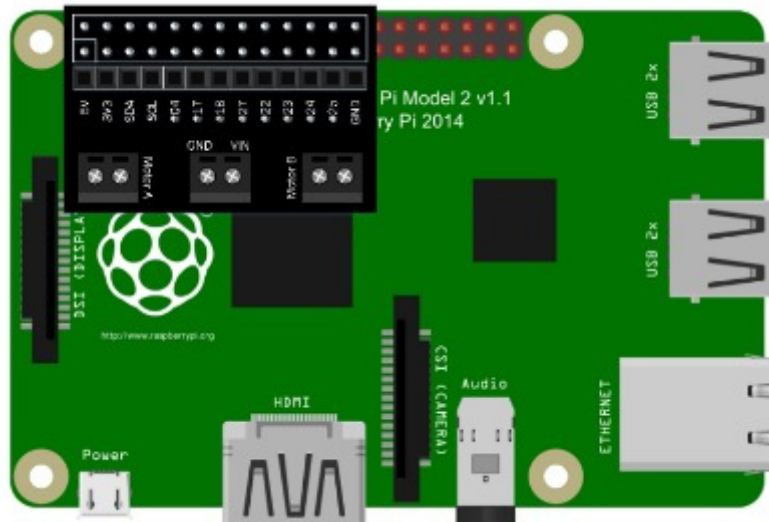
A motor can be driven forwards or backwards depending on which way around current flows through it. However, it would be awkward to have to rewire a motor, every time you want to change the direction it spins. To overcome this issue, motor controller boards include an H bridge. An H bridge uses 4 transistors to allow digital control of which way current flows through the motor. Most H bridges also contain *flyback diodes*. A flyback diode prevents the voltage spike that is generated by the motor when it is no longer powered (but still spinning) from damaging delicate electronics.



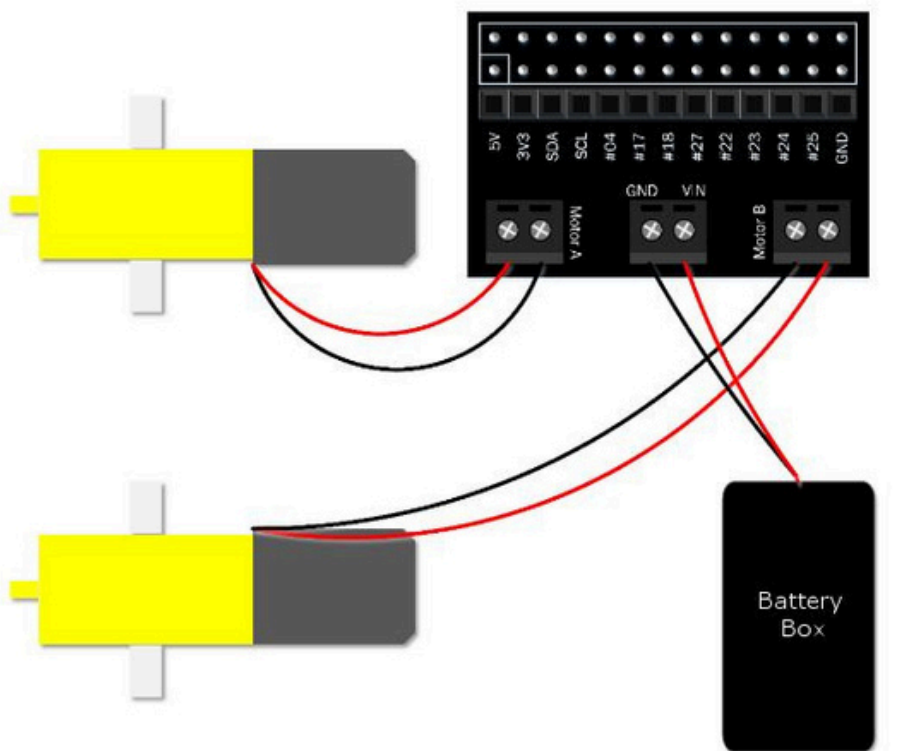
Wiring

You'll need to wire up two motors and your battery pack using the motor controller.

- ✓ With your Pi switched off, mount your motor controller board on the GPIO pins:



✓ Connect a battery pack to the power ports of the motor controller, connecting the positive (red) battery wire to the positive (+) power terminal on the motor controller, and the negative (black) battery wire to the negative (-) power terminal on the motor controller, and connect two motors:



✓ You'll need to know which GPIO pins your motor controller uses. Refer to the board's documentation. This will usually be described as Motor A and Motor B, or MA1, MA2, MB1, and MB2. Make a note of these pin numbers. If you're not sure which is which, you can investigate this next.

Motor class

You can use the built-in **Motor** class to control motors.

- ✓ Import the **Motor** class:

```
from gpiozero import Motor
```

Now create a **Motor** instance using the pin numbers for each motor:

```
motor1 = Motor(4, 14)
motor2 = Motor(17, 27)
```

- ✓ Note: to make it easier to see which pin is which, you can use `Motor(forward=4, backward=14)` for future reference.
- ✓ Now drive one of the motors forward using the following code:

```
motor1.forward()
motor2.backward()
motor1.forward(0.5)
motor2.backward(0.5)
```

The **Motor** class also allows you to reverse the motor's direction. Try using this loop:

```
motor1.forward()
motor2.backward()
while True:
    sleep(5)
    motor1.reverse()
    motor2.reverse()
```

- ✓ This will make the motors spin in opposite directions, then switch every five seconds. Press **Ctrl + C** to exit the loop.
- ✓ Now stop the motors:

```
motor1.stop()
motor2.stop()
```

LAB TASKS

1. Simulate a traffic light using 3 LEDs (Red, Yellow, Green) with appropriate delays.
2. Build a simple motion detection alarm system using PIR sensor, LED, and buzzer.
3. Read temperature and humidity from DHT11 and log data with timestamp.
4. Use a potentiometer to control the angle of a servo motor (simulate fan speed/angle).
5. Create a simple lock system with 4 push buttons (representing digits 1–4).

Task:

1. Simulate a traffic light using 3 LEDs (Red, Yellow, Green) with appropriate delays.

Traffic Light using 3 LEDs:

Code:

```
from gpiozero import LED
from time import sleep

red = LED(17)
yellow = LED(18)
green = LED(27)

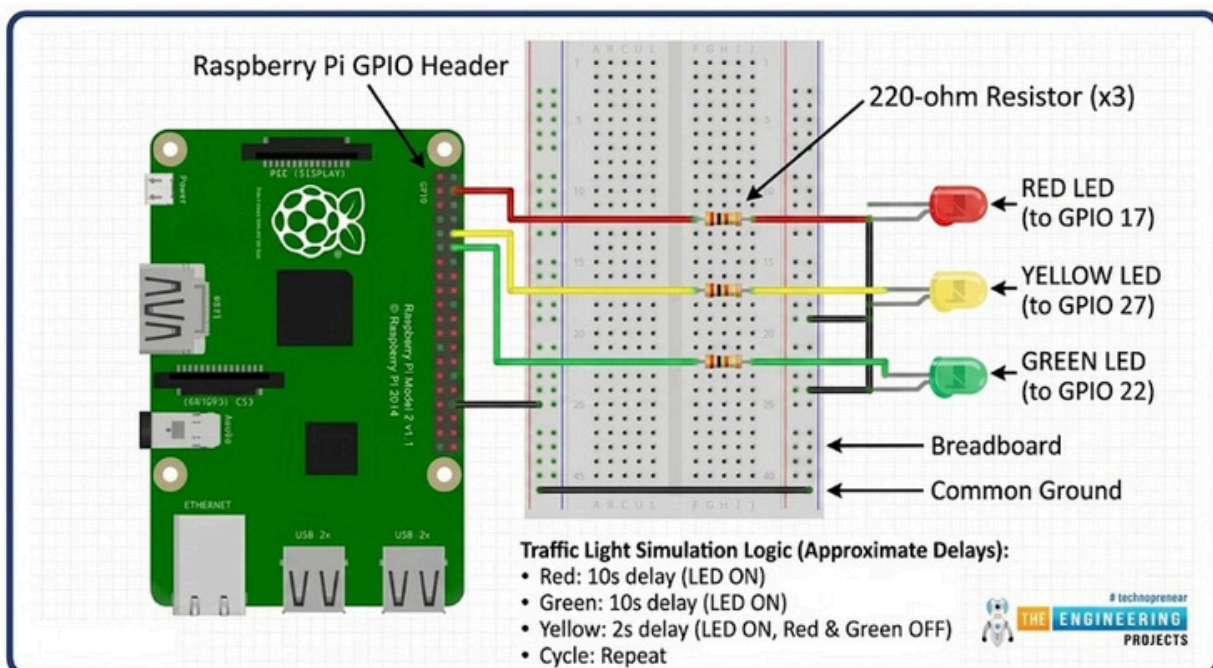
while True:

    red.on()
    sleep(5)
    red.off()

    yellow.on()
    sleep(2)
    yellow.off()

    green.on()
    sleep(5)
    green.off()
```

Block Diagram:



Expected Output

Red LED ON (Stop)
Yellow LED ON (Wait)
Green LED ON (Go)
Cycle repeats continuously

Task:

2. Build a simple motion detection alarm system using PIR sensor, LED, and buzzer.

Code:

```
from gpiozero import MotionSensor, LED, Buzzer

pir = MotionSensor(4)
led = LED(17)
buzzer = Buzzer(18)

while True:

    pir.wait_for_motion()

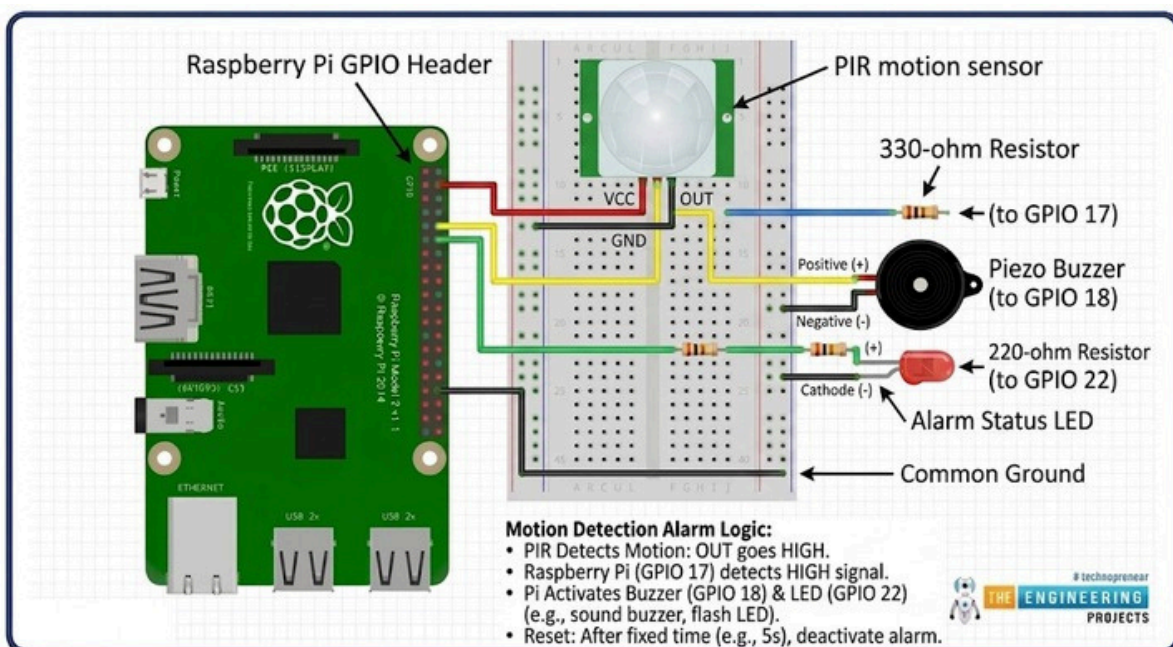
    led.on()
    buzzer.on()

    print("Motion Detected")

    pir.wait_for_no_motion()

    led.off()
    buzzer.off()
```

Block Diagram:



Expected Output:

When motion is detected:

LED turns ON

Buzzer sounds

"Motion Detected" displayed

When no motion:

LED OFF

Buzzer OFF

Task:

3. Read temperature and humidity from DHT11 and log data with timestamp.

DHT11 Temperature & Humidity Logger

Code:

```
import Adafruit_DHT
import time
from datetime import datetime

sensor = Adafruit_DHT.DHT11

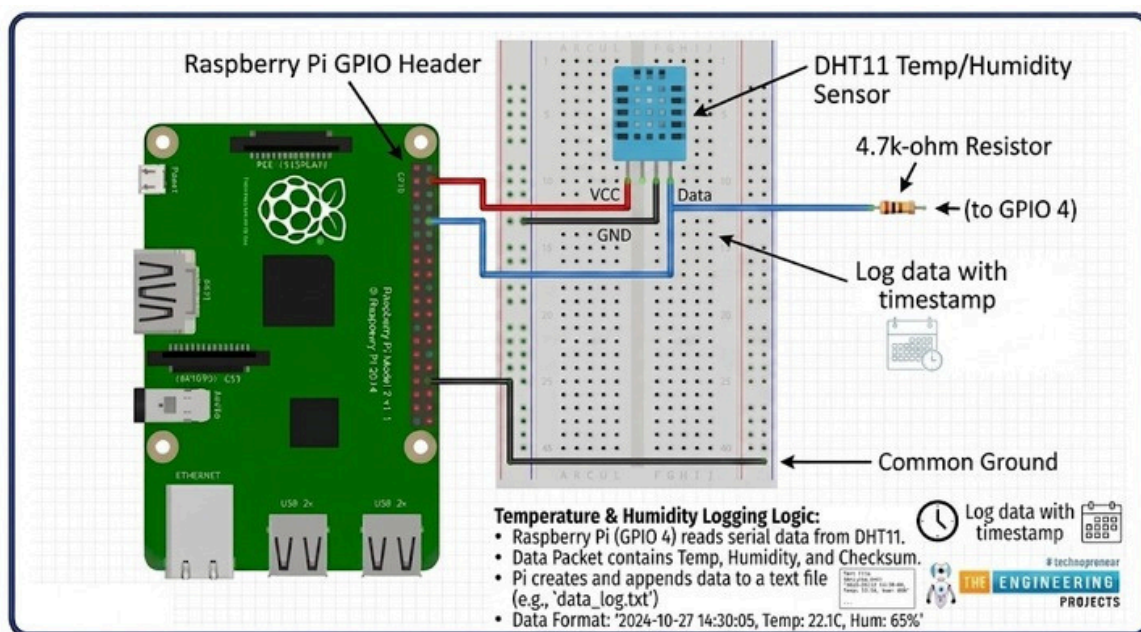
while True:

    humidity, temperature = Adafruit_DHT.read_retry(sensor, 4)

    print(datetime.now(),
          "Temp =", temperature,
          "Humidity =", humidity)

    time.sleep(5)
```

Block Diagram:



Expected Output:

Rotate potentiometer
clockwise:
Servo rotates right

Rotate potentiometer anti-
clockwise:
Servo rotates left

Task:

5. Create a simple lock system with 4 push buttons (representing digits 1–4).

4-Button Lock System

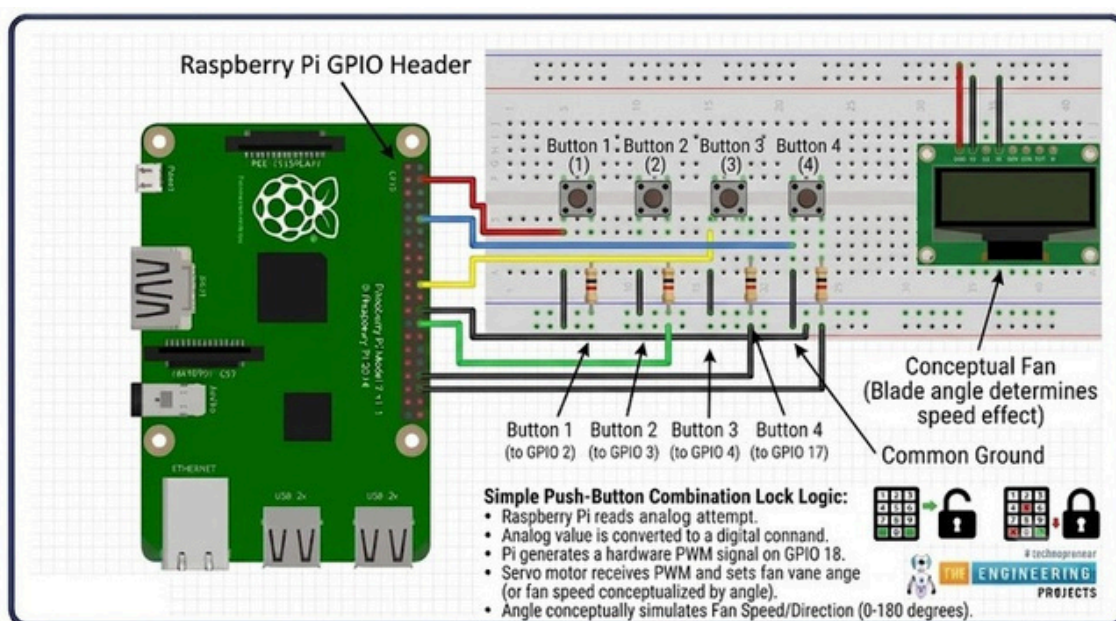
Code:

```
password = "1234"

entered = input("Enter Password: ")

if entered == password:
    print("Lock Opened")
else:
    print("Access Denied")
```

Block Diagram:



Expected Output:

2025-05-10 10:00:01

Temp = 28°C

Humidity = 65%

2025-05-10 10:00:06

Temp = 28°C

Humidity = 66%

Task:

4. Use a potentiometer to control the angle of a servo motor (simulate fan speed/angle).

Potentiometer Controls Servo Motor

Code:

```
from gpiozero import MCP3008, AngularServo
from time import sleep

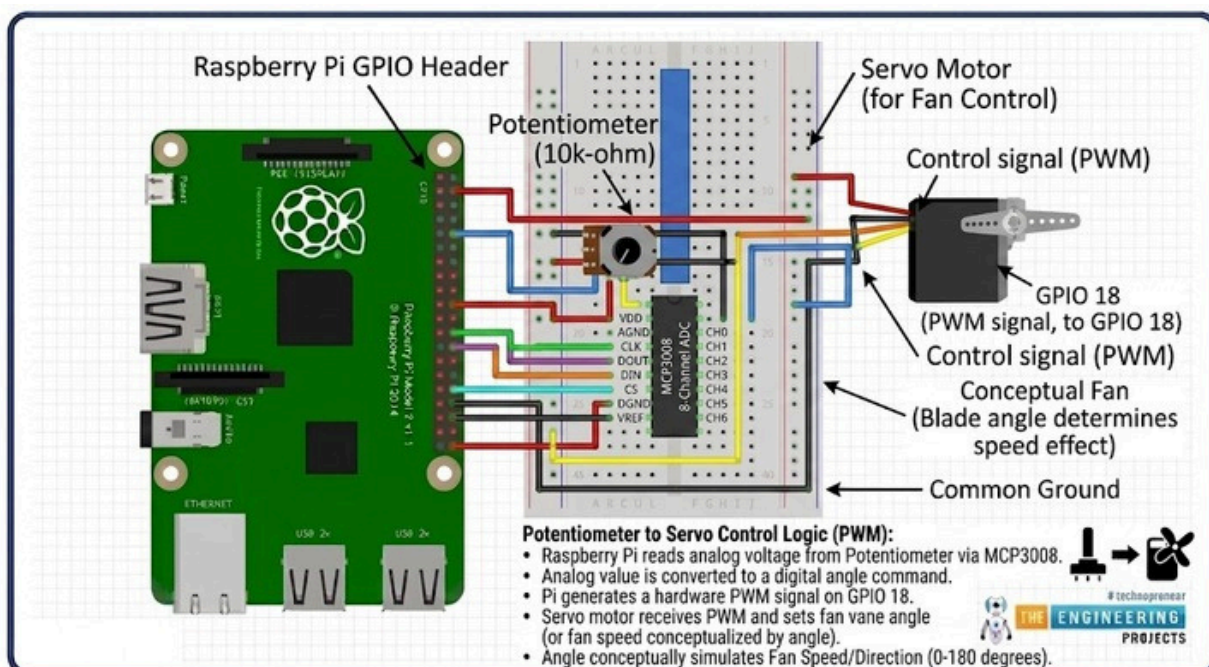
pot = MCP3008(0)
servo = AngularServo(17)

while True:

    servo.angle = (pot.value * 180) - 90

    sleep(0.1)
```

Block Diagram:



Expected Output:

Use buttons to enter:

Button1 = digit 1

Button2 = digit 2

Button3 = digit 3

Button4 = digit 4

Correct sequence:

1 → 2 → 3 → 4

Output:

Lock Opened

Wrong sequence:

Access Denied

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 5

Physical Computing with Raspberry PI

Name: Bushra Nazir **Reg. No.:** 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) <u>PSYCHOMOTOR</u> (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Imcomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
	SUB-TOTAL (PT)	7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) <u>COGNITIVE</u> (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
	SUB-TOTAL (CT)	1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) <u>AFFECTIVE</u> (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
	SUB-TOTAL (AT)	2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) : 10

Marks Obtained (PO + CO + AO) :

Instructor